CS 647. HW 3 Xiyao Long

Run:

Use IntelliJ to run the program, the main function is in DiningHakkersOnFsm.java
The .java files are all in ./src/main/java/sample.fsm

Update on the Raft system:

Fixed possible bugs in the leader election. In program running, a leader is generally selected on the first term. This is not a bug.

Log implementation is attempted using java serialization and logger with file handler. The serialization should be better because it allows latter change of log entries more easily. In the submitted version both logging method are commented out because it may break the system.

The function of applying to state machine is implemented.

On top of the raft system, there are several constraints in implementing the ticketing system:

1. The # of available tickets never go under zero:
The ticket sell write should only be applied to the leader, this indicates if the number of tickets after the write is below zero, then such request should be rejected, else if it's above zero, then write can be proceed and an acknowledgment message should be sent back to the client to confirm that the purchase is done.

2. The consistent read/write should be as not very frequent to give good performance to the system.
Any frontend get the number of tickets by quering(message exchange) its corresponded node. The result is an unstable read, meaning it's the result of applying both committed and uncommited log entries to the state machine of any node.

Ticketing System Implementation:

Each frontend has a corresponding backend node. It send its unstable read requests and consistent write request to its backend node. The backend node will return its statemachine upon read request, but will reroute the write request to the raft leader.

The leader, upon receiving the write request, will compare the requested ticket number to the ticket remaining in its statemachine. If there's enough ticket, it will send

back an acknowledgement message to the corresponding frontend, else it will send back a failure message notifying there's not enough ticket left.

To reduce the number of consistent write operation, each client frontend will reserve a certain number of tickets(20) from the backend nodes leader whenever a customer order n tickets exceeding the number of reserved tickets. Any buy request will be process inside the client frontend without consistent write operation unitll is message to the backend node leader is necessary.

Result and Bugs:

Some node seem to do not apply entry to stjatemachine successfully while the other nodes have the entries applied(Figure 2).

The commit index sometimes do not update and always remain zero, However when I rerun the commit indexes update (Figure 1). The success rate is probably 2 or 3 out of 10 times.

The application can write to the leader and change its available number of tickets. The number never goes under zero.(Figure 3 & 4). The Client also would buy ticket without giving writes to backend nodes, but just from the reserved 20 tickets upon each leader write request(Figure 3).

Figure 1.

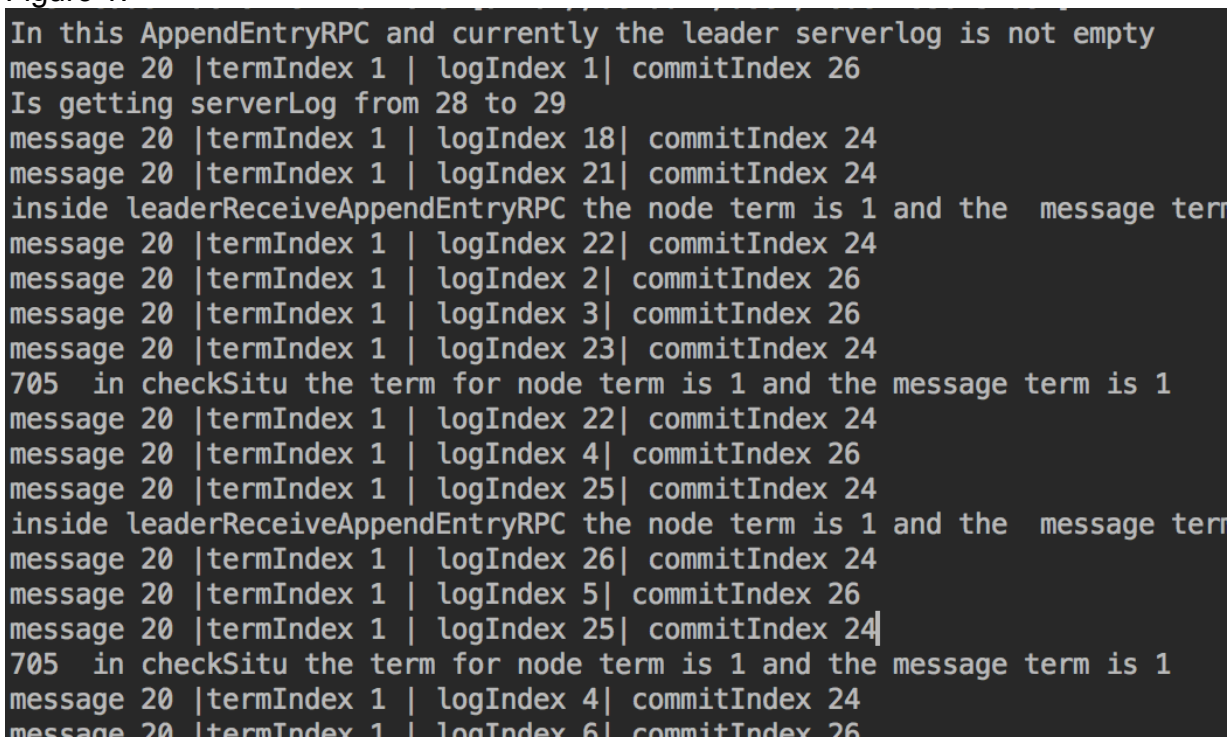


Figure 2.

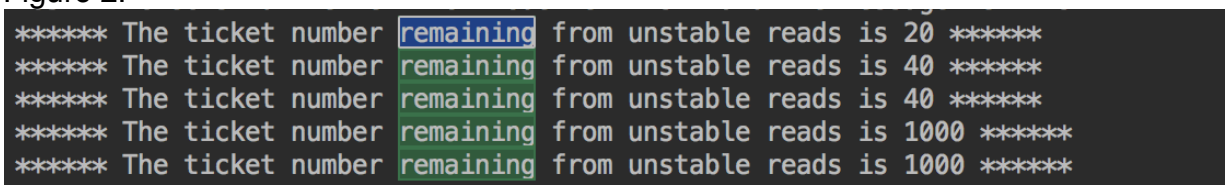


Figure 3.

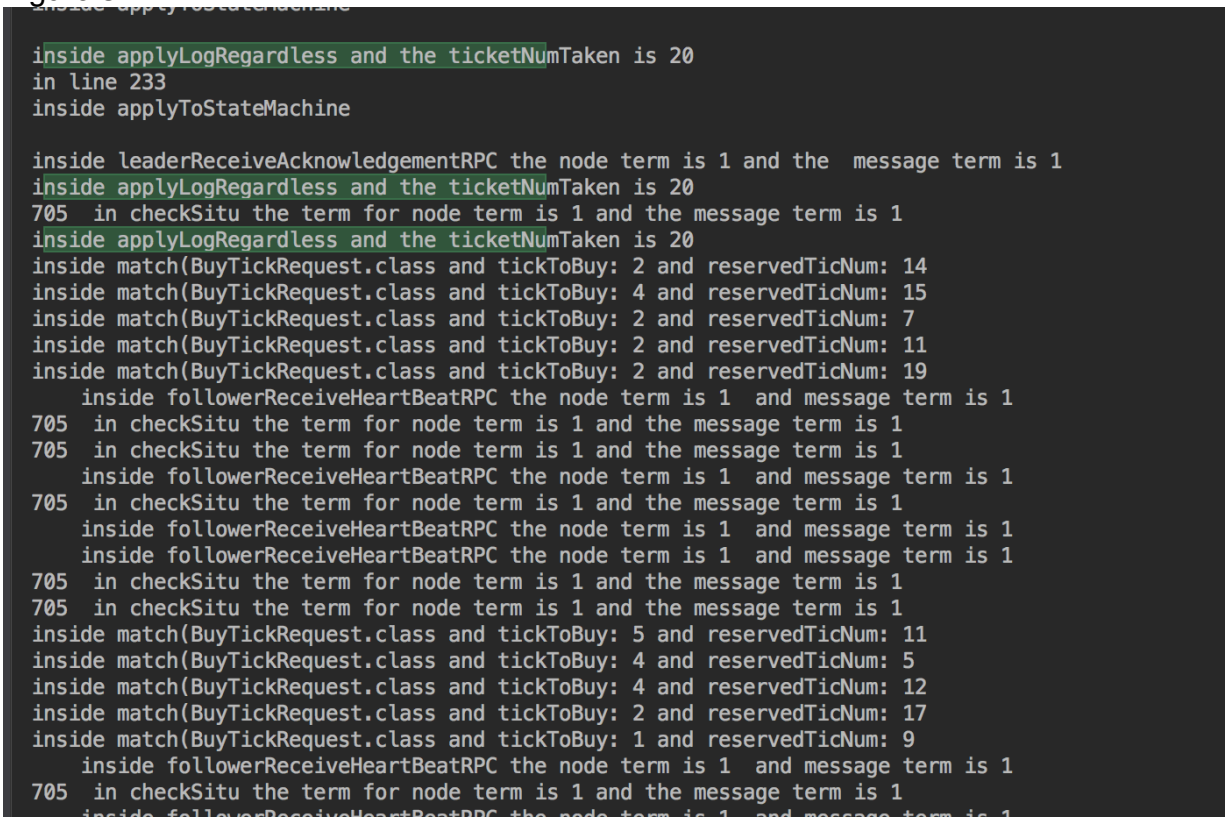
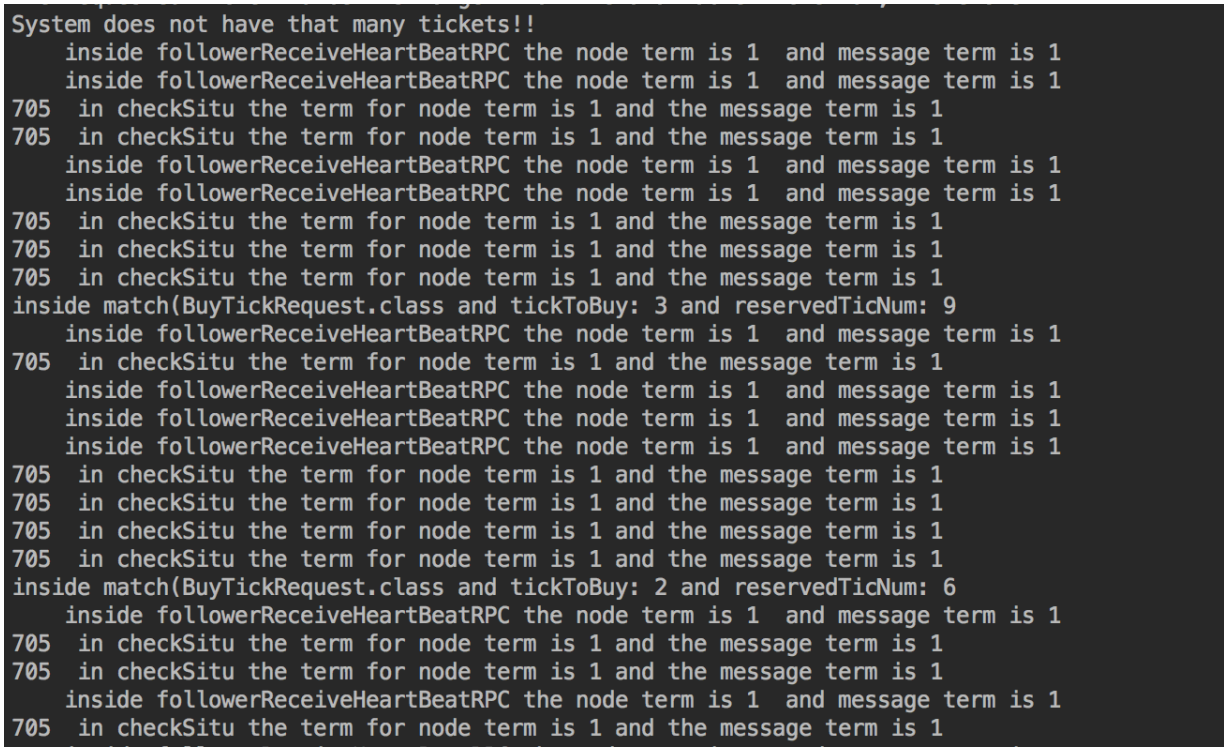


Figure 4.



Reflection:

- How does what you've implemented compare to how you might implement similar functionality on top of Bayou? What parts of this would become easier on Bayou (e.g., with custom conflict resolvers), and what might become harder?

If this application is implemented on top of Bayou, as Bayou is a weakly consistent protocol. However, I can still implement the system in a similar way compared to raft. The read access will still be “unstable” reads and the values are those returned from any backend server node the frontend communicates. We can not enforce a consistent write operation as we don’t really know how long can we wait until the system reach final consistency. However, this is not much of a problem if we only allow non-primary write operation when ticket number is greater than certain threshold, chosen based on our understanding of how the system behaves in terms of how the actual available tickets changes. The Bayou protocol would resolve any discrepancies of such write operations between different backend nodes.

When the unstable read is under such threshold, the frontend will communicates with only the primary server, so that any of its write operation will be committed immediately and reduce the chance of letting the number goes below zero.

- What additional guarantees does this approach have compared to implementing the functionality on top of CRDTs like the bounded counter CRDT? What advantages does having a single log bring over using CRDTs? (Note: you don't have to read that paper to answer this question.)

The bounded counter CRDT enforces the number of operations/some threshold on the value before synchronizing the value with other servers. But Bayou allow some global invariant as constraint which is not possible for CRDT.(1) As for the application we implemented, it enforce consistent write that is not the feature of CRDT, the remaining ticket will never go under zero.

- What are the disadvantages of using the approach taken here, compared to using CRDTs? What sort of reliability advantages might CRDTs bring to this system, that we've given up by using a single log?

Using raft system and only write to the leader is in a way restricting the number of write request the system can process in a limited time. If the client requests amount is large, it can cause latency problem.

Resources:

Akka documentation java code/ tutorial.

Reference

1. Kleppmann, et al. “A Conflict-Free Replicated JSON Datatype.” ArXiv.org, 15 Aug. 2017, arxiv.org/abs/1608.03960v3.